

ENLANG FOR DEVELOPERS

The Definitive Guide to Building Full-Stack Software with Natural English

Author: Spandan Prayas Patra

Version: 2.0.0 — Enterprise Specification Edition

Publisher: Universal EnLang Foundation Press

"Programming should be as clear, expressive, and frictionless as human thought. EnLang makes natural English a first-class, production-grade programming language across the entire software stack."

— Spandan Prayas Patra

TABLE OF CONTENTS

- Chapter 1: Introduction & Philosophical Architecture
 - Chapter 2: Variables, Optional Types & Expression Syntax
 - Chapter 3: The 3-Level Native Interactivity Architecture
 - Chapter 4: Control Flow & Iteration
 - Chapter 5: Functions, Classes, Interfaces & Async / Await
 - Chapter 6: Pattern Matching & Decision Engine
 - Chapter 7: Exception Handling & Error Control
 - Chapter 8: Built-in Collections — Lists, Arrays, Dictionaries, Sets
 - Chapter 9: String Manipulations, Math & DateTime Primitives
 - Chapter 10: File I/O, System Operations & Security
 - Chapter 11: Frontend Structural Markup (`.enlgf` → HTML5)
 - Chapter 12: Styling & Design Systems (`.enlgd` → CSS3)
 - Chapter 13: Client-Side Scripting (`.enlgs` → JavaScript ES6+)
 - Chapter 14: Database Schemas & Queries (`.enlgdb` → SQL)
 - Chapter 15: Native NLP & AI Primitives
 - Chapter 16: EnLang Web Server & Multi-File Architecture
 - Chapter 17: Canonical Grammar Rules, Order & Syntax Layout Specification
 - Chapter 18: Complete Production Case Study — Lumina Workspace
 - Appendix A: Universal Natural Syntax Cheatsheet
 - Appendix B: CLI & EnLang Package Manager (EPM) Manual
-

CHAPTER 1: INTRODUCTION & PHILOSOPHICAL ARCHITECTURE

1.1 The Vision of Natural Programming

For over seven decades, software development has required human engineers to adapt their thoughts to artificial syntaxes, cryptic symbols (`{}`, `;`, `=>`, `::`), and unforgiving compiler rules.

EnLang was designed by **Spandan Prayas Patra** to invert this paradigm: to allow developers to write application logic in **pure, clear, expressive natural English**, while maintaining **100% deterministic compilation** into battle-tested native target languages (Python 3, HTML5, CSS3, JavaScript ES6+, SQL).

EnLang is **not** a pseudo-code generator or an LLM wrapper. It is a **deterministic, multi-target compiler and runtime engine**.

CHAPTER 2: VARIABLES, OPTIONAL TYPES & EXPRESSIONS

2.1 Natural Expressive Flexibility

EnLang supports multiple natural syntaxes for variable assignment:

```
set score to 100
let score = 100
store 100 in score
score is 100
```

2.2 Optional Type System (v2.0 Specification)

Type annotations are completely optional:

```
define number age as 20
define decimal price as 99.50
define text username as "Spandan"
define boolean isActive as true
define list fruits as ["Apple", "Banana"]
define array tags as ["ai", "nlp"]
define dictionary profile as {"name": "Spandan"}
define set unique_ids as {101, 102}
```

CHAPTER 3: THE 3-LEVEL NATIVE INTERACTIVITY ARCHITECTURE (v2.0)

Level 1: Pure EnLang (Default)	→ set area to width times height
Level 2: Inline Native Marker	→ set root to @python(math.sqrt(144))
Level 3: Multi-Line Native Block	→ python: ... end python

CHAPTER 4: CONTROL FLOW & ITERATION

```
if score is greater than 90 then:
    display "Grade: A+"
otherwise if score is greater than 80 then:
    display "Grade: A"
else:
    display "Grade: B"

repeat 5 times do:
    display "Processing batch item..."

for each user in users do:
    display "User: " plus user
```

CHAPTER 5: FUNCTIONS, CLASSES, INTERFACES & ASYNC / AWAIT

5.1 Enterprise Interfaces & Implements (v2.0)

```
interface Authenticatable:
    function login(credentials)
    function logout()
end interface

class UserSession implements Authenticatable:
    function login(credentials):
        display "User authenticated"

    function logout():
        display "Session closed"
end class
```

5.2 Async & Await Primitives (v2.0)

```
import module asyncio

async function fetch_user_data(user_id):
    sleep 0.1 seconds
    return "Profile Data for ID " plus str(user_id)

async function main():
    set profile to await fetch_user_data(101)
    display profile

asyncio.run(main())
```

CHAPTER 6: PATTERN MATCHING (match / case / default)

```
set role to "admin"

match role:
    case "admin":
        display "Full System Access"
    case "editor", "author":
        display "Content Modification Access"
    case is greater than 50:
        display "Score Requirement Met"
    default:
        display "Guest Access Only"
end match
```

CHAPTER 7: EXCEPTION HANDLING (try / except / finally)

```
try:
    set result to 100 divided by 0
except:
    display "Caught runtime exception"
finally:
    display "Cleanup block executed"

raise ValueError with message "Input out of bounds"
throw error "Authentication failure"
```

CHAPTER 17: CANONICAL GRAMMAR RULES, ORDER & SYNTAX LAYOUT SPECIFICATION (v2.0)

To prevent errors caused by arbitrary English phrasing, EnLang enforces strict **Canonical Grammar Rules, Precedence, and Structural Layout Boundaries**.

17.1 Block Indentation & Scope Rules

- **Indentation:** All indented blocks (`if`, `for`, `while`, `function`, `class`, `match`, `try`) MUST use **4 spaces per level**.
- **Colon Signifier:** Every block header MUST terminate with a colon (`:`).
- **Closing Tags:** Explicit block terminators (`end match`, `end interface`) MUST align with the starting indentation column.

Correct Block Layout

```
function process(items):  
    for each item in items do:  
        if item is greater than 0 then:  
            display item
```

Incorrect Block Layout (WILL CAUSE SYNTAX ERROR)

```
function process(items)    # ERROR: Missing trailing colon ':'  
display items              # ERROR: Must use 4 spaces, not 2
```

17.2 Valid vs. Invalid Natural Phrase Variants

Variable Declarations

- **Valid:** `set x to 10`, `let x = 10`, `store 10 in x`, `define number x as 10`
- **Invalid:** `assign 10 to variable x` (Syntax Error: use `set` or `store`)

Comparison Operators

- **Valid:** `is equal to`, `is not equal to`, `is greater than`, `is less than`, `is greater than or equal to`
- **Invalid:** `is bigger than` (Syntax Error: use `is greater than`)
- **Invalid:** `is same as` (Syntax Error: use `is equal to`)

Arithmetic Operators

- **Valid:** `plus`, `minus`, `times`, `divided by`, `modulo`, `power of`
- **Invalid:** `add x and y` in expressions (Use `x plus y` for inline expressions; `add x to list` is reserved for collection append).

17.3 Statement Ordering & Module Import Precedence

- **Top-of-File Imports:** All `import module <name>` statements MUST be placed at the top of the file before function or class definitions.
- **Interface Definitions Before Implementation:** Interfaces MUST be declared before any class using `implements <Interface>`.
- **Base Class Definitions Before Subclasses:** Parent classes MUST be defined before child classes using `extends <BaseClass>`.

17.4 Operator Precedence Table

Precedence	Natural Operator	Native Equivalent	Association
1 (Highest)	<code>()</code>	Grouping	Left-to-Right
2	<code>power of</code>	<code>**</code>	Right-to-Left
3	<code>times</code> , <code>divided by</code> , <code>modulo</code>	<code>*</code> , <code>/</code> , <code>%</code>	Left-to-Right
4	<code>plus</code> , <code>minus</code>	<code>+</code> , <code>-</code>	Left-to-Right
5	<code>is equal to</code> , <code>is greater than</code> , etc.	<code>==</code> , <code>></code> , etc.	Left-to-Right
6	<code>not</code>	<code>not</code>	Right-to-Left
7 (Lowest)	<code>and</code> , <code>or</code>	<code>and</code> , <code>or</code>	Left-to-Right

CHAPTER 18: PRODUCTION CASE STUDY — LUMINA WORKSPACE

`server.enlg`

```
import module json

set app_name to "Lumina Workspace"
set version to "2.0.0"

display "Booting " plus app_name plus " v" plus version

start web server on port 8000
```

APPENDIX A: UNIVERSAL SYNTAX REFERENCE

Category	EnLang Syntax Example	Native Transpilation
Level 1 (Pure)	set area to width times height	area = width * height
Level 2 (Marker)	set val to @python(math.sqrt(25))	val = math.sqrt(25)
Level 3 (Block)	python: ... end python	Verbatim Block Execution
Interface	interface Authenticatable: ... end interface	Abstract Contract Class
Implements	class UserSession implements Authenticatable:	Multi-Inheritance Class
Async Func	async function fetch_data():	async def fetch_data():

APPENDIX B: CLI & PACKAGE MANAGER (EPM) MANUAL

enlang run app.enlg	# Compiles & executes backend logic
enlang server --port 8000	# Launches zero-config HTTP host server
enlang add py:requests	# Installs Python PyPI package
enlang add web:chart.js	# Installs Web NPM package

Copyright © 2026 Spandan Prayas Patra. All rights reserved.
Published under the Open EnLang Specification License.